Playwright Notes –> Part-1

1.What is Playwright? Advantages and Limitations

Playwright is a modern web automation testing framework developed by Microsoft. It is used for end-to-end testing of web applications. Playwright allows testers to automate browsers and simulate actions like clicking, typing, navigation, and validating UI behavior. It supports fast and reliable automation for modern web applications.

Advantages of Playwright

- Supports multiple browsers: Chrome, Firefox, and WebKit
- Very fast execution with built-in parallel testing
- Auto-wait feature reduces flaky test failures
- Supports mobile device emulation
- Can capture screenshots and videos automatically
- Supports multiple programming languages: JavaScript, Java, Python, C#, TypeScript
- No need to download separate drivers like Selenium
- Supports headless and headed browser execution
- Built-in test runner and reporting tools
- Works well with modern frameworks like React, Angular, and Vue
- Strong element handling and smart selectors
- Good debugging tools with trace viewer

Limitations of Playwright

- Newer framework compared to Selenium
- Smaller community and learning resources
- Limited third-party integrations
- Less adoption in older enterprise projects
- Fewer ready-made plugins compared to Selenium ecosystem
- Some teams prefer Selenium due to long industry history

2.Playwright Architecture

Playwright architecture has three main layers.

Test Code

This is where we write automation scripts. It contains test steps and validations written in languages like Java, Python, or JavaScript.

Playwright Library

This layer connects the test code to the browser. It sends commands, handles waiting, and controls browser actions.

Browsers

Playwright runs tests directly on real browsers like Chrome, Firefox, and WebKit.

Flow:

Test Code → Playwright Library → Browser

3. Playwright vs Cypress vs Selenium WebDriver

Playwright, Cypress, and Selenium are popular automation tools used for web testing. Each tool has its own strengths

Playwright

Supports multiple browsers like Chrome, Firefox, and WebKit. It is fast and supports parallel execution. Good for modern web applications and cross-browser testing.

Cypress

Mainly focused on UI testing. Easy to set up and good for frontend testing. However, it has limited browser support compared to Playwright and Selenium.

Supports almost all browsers. It is older, stable, and widely used in industry. It has a large community and strong ecosystem, but execution can be slower compared to Playwright.

4.Software Requirements

To work with Playwright, the following software is required:

Node.js

Used to install Playwright and manage project dependencies.

VS Code Editor

Used to write and manage automation scripts.

Browsers

Supported browsers like Chrome, Edge, and Firefox are required to run tests.

5.Download and Install Node.js and VS Code

- Download Node.js from https://nodejs.org
- Download VS Code from https://code.visualstudio.com
- Install both applications by following setup instructions

Install Playwright using VS Code

- Open VS Code
- Open the terminal inside VS Code
- **Run:**

  npm init playwright@latest
- Follow the on-screen instructions

Default Playwright Folder Structure

- tests – contains test scripts
- playwright.config.ts – configuration file
- test-results – stores reports and screenshots
- node_modules – installed libraries

Playwright Test Explorer

- VS Code extension
- Helps filter, run, and debug tests from UI

Setup Playwright using Command Prompt

- Open Command Prompt or terminal
- **Run:**

  npm init playwright@latest

Record Playwright Test using VS Code

- **Run:**

  npx playwright codegen https://example.com
- Perform actions and Playwright generates code automatically

Generating HTML Test Report

- After running tests, run:

  npx playwright show-report

Run Tests on Chrome, Firefox and WebKit

npx playwright test --project=chromium
npx playwright test --project=firefox
npx playwright test --project=webkit

Generate Playwright HTML Report

npx playwright show-report

Common Terminologies

- Test – single test case
- Fixture – setup or environment for a test
- Locator – identifies web elements

- Context – separate browser session
- Headless – browser runs without UI

First Playwright Test Example

```
test('Verify page title', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle('Example Domain');
});
```

Pick Locator in VS Code

- Use codegen or selector tool to capture element locators

Record at Cursor

- Record steps and insert code at cursor using Playwright extension

Run Specific Spec File

```
npx playwright test tests/example.spec.ts
```

Run Tests in Headless Mode

```
npx playwright test --headless
```

Run Tests in Headed Mode

```
npx playwright test --headed
```

Run Tests on Different Browsers using CMD

```
npx playwright test --project=chromium
npx playwright test --project=firefox
npx playwright test --project=webkit
```

Codegen – Record and Play Test

```
npx playwright codegen https://example.com
```

- Records actions and generates automation code

6.Playwright Locator Strategies

Playwright provides many ways to identify elements. These locators help create stable and readable automation scripts.

By Role

Description:

Finds elements based on accessibility roles like button, link, textbox. Recommended for modern apps.

Example:

```
await page.getByRole('link', { name: 'value' }).click();
```

By Label

Description:

Finds input fields using label text. Useful for forms.

```
await page.getByLabel('value', { exact: true }).fill('value');
```

By Alt Attribute

Description:

Locates images using alt text.

```
await page.getByAltText('alt value').click();
```

By Test ID

Description:

Uses custom testing attributes added by developers. Very stable for automation.

```
await page.getByTestId('value').fill('text');
```

By Text

Description:

Finds elements using visible text.

```
await page.getByText('text').click();
await page.getByText('text', { exact: true }).click();
```

By Placeholder

Description:

Finds input fields using placeholder text.

```
await page.getByPlaceholder('value').click();
```

By Title

Description:

Finds elements using title attribute.

```
await page.getByTitle('title text').click();
```

By ID

Description:

Uses element id attribute. Fast and reliable if unique.

```
await page.locator('#username').fill('admin');
```

By Class

Description:

Finds elements using class name.

```
await page.locator('.login-btn').click();
```

By Attribute

Description:

Uses any custom attribute.

```
await page.locator('[data-test="submit"]').click();
```

CSS Selector

Description:

Uses CSS patterns to locate elements.

```
await page.locator('tag[attr="value"]').click();
```

XPath Locator

Description:

Uses XPath expressions. Helpful when other locators are not available.

```
await page.locator("//*[@attr='value']").click();
```

Chained Locator

Description:

Searches inside another element.

```
await page.locator('#form').locator('button').click();
```

Filter Locator

Description:

Filters elements based on text or condition.

```
await page.locator('li').filter({ hasText: 'Item' }).click();
```

Nth Element Locator

Description:

Selects element by index.

```
await page.locator('.item').nth(0).click();
```

First / Last Locator

Description:

Selects first or last matching element.

```
await page.locator('.item').first().click();
```

```
await page.locator('.item').last().click();
```

7.How to Capture Screenshots in Playwright

In Playwright, we can capture the screenshots of a specific element, the visible page, or the full page.

Element Screenshot

Description:
Captures a screenshot of a selected element only.
Example:

```
const element = page.locator('#logo');
await element.screenshot({ path: 'element-screenshot.png' });
```

Explanation:
Only the chosen element is saved as an image.

Page Screenshot

Description:
Captures the currently visible part of the page.
Example:

```
await page.screenshot({ path: 'page-screenshot.png' });
```

Explanation:
Only the visible screen area is captured.

Full Page Screenshot

Description:
Captures the entire scrollable page.
Example:

```
await page.screenshot({
  path: 'fullpage-screenshot.png',
  fullPage: true
});
```

Explanation:
Captures the full-page including content below the scroll.

8.How to Add Screenshots to Playwright HTML Test Report

Playwright can automatically attach screenshots to the HTML report using configuration.

Screenshot Only When Test Fails

Description:
Automatically captures screenshots when a test fails.
Example (playwright.config.ts):

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    screenshot: 'only-on-failure'
  },
  reporter: [['html']]
});
```

Explanation:
Screenshots appear in the report only for failed tests.

Screenshot for Passed and Failed Tests

Description:
Captures screenshots for every test.
Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    screenshot: 'on'
  },
  reporter: [['html']]
});
```

Explanation:
Screenshots are attached for both passed and failed tests.

Viewing Screenshots in Report

Run:

```
npx playwright show-report
```

This opens the HTML report with attached screenshots.

9.How to Implement Hooks in Playwright

Hooks are used to run setup and cleanup code before or after tests. They help avoid repeating common steps.

beforeAll()

Description:
Runs once before all tests in a test file. Used for global setup.

beforeEach()

Description:
Runs before every test. Commonly used to open a page or login.

afterEach()

Description:

Runs after every test. Used for cleanup steps.

afterAll()

Description:

Runs once after all tests are completed. Used for final cleanup.

Example

```
test.beforeEach(async ({ page }) => {
  await page.goto('https://example.com');
});
```

Explanation:

This hook opens the website before each test runs.

10.Handling Dropdown List in Playwright

Playwright provides built-in methods to select values from dropdown lists.

Select Dropdown by Value

Description:

Selects an option using the value attribute.

Example:

```
await page.selectOption('#dropdown', { value: 'Feb' });
```

Select Dropdown by Visible Text

Description:

Selects an option using the displayed label.

Example:

```
await page.selectOption('#dropdown', { label: 'March' });
```

Example Dropdown Options

Jan, Feb, Mar, Apr, May, Jun,Jul, Aug, Sep, Oct, Nov, Dec

11.Switching into IFrames in Playwright

Description:

Used to interact with elements inside an iframe.

Example:

```
const frame = page.frame({ name: 'frameName' });
await frame.click('#inside-frame-button');
```

12.Mouse Actions in Playwright

Description:

Playwright supports common mouse operations.

Drag and Drop Elements

Description:

Moves an element from one location to another.

```
await page.dragAndDrop('#source', '#target');
```

Left click:

```
await page.mouse.click(100, 200);
```

Right click:

```
await page.mouse.click(100, 200, { button: 'right' });
```

Hover:

```
await page.hover('#element');
```

Double click:

```
await page.dblclick('#element');
```

13.Keyboard Actions in Playwright

Description:

Simulates keyboard input.

Press Enter:

```
await page.keyboard.press('Enter');
```

Press Tab:

```
await page.keyboard.press('Tab');
```

Press Delete:

```
await page.keyboard.press('Delete');
```

Press Control + A:

```
await page.keyboard.press('Control+A');
```

14.Handling Date Picker in Playwright

Description:

Dates can be entered directly or generated dynamically.

Hardcoded date:

```
await page.fill('#date-input', '2024-12-25');
```

Today's date:

```
const today = new Date().toISOString().split('T')[0];
await page.fill('#date-input', today);
```

Past date:

```
const past = new Date();
past.setDate(past.getDate() - 5);
await page.fill('#date-input', past.toISOString().split('T')[0]);
```

Future date:

```
const future = new Date();
future.setDate(future.getDate() + 10);
await page.fill('#date-input', future.toISOString().split('T')[0]);
```

15.Assertions in Playwright

Description:

Assertions are used to verify expected results.

Common assertions:

- toBeEditable()
- toBeVisible()
- toBeEnabled()
- toBeEmpty()
- toBeDisabled()
- toHaveURL()
- toHaveTitle()
- toHaveText()
- toHaveCount()

Example:

```
await expect(page.locator('#username')).toBeEditable();
await expect(page.locator('#submit')).toBeVisible();
```

Watch Mode

Description:

Automatically reruns tests when code changes.

Command:

```
npx playwright test --watch
```

Playwright UI Mode

Description:

Helps debug tests with step-by-step execution, screenshots, and logs.

Command:

npx playwright test --ui

Trace Viewer

Description:

Shows test steps, console logs, network activity, and screenshots.

Command:

npx playwright show-trace trace.zip